

/*

- TRABALHO PRÁTICO: ULA DE 4 BITS + ARDUINO
- Mapeamento da Memória Interna (Vetor de 100 posições):
- memoria[0] -> PC (Program Counter): Posição da memória com a instrução atual (inicia em 4).
- memoria[1] -> W (Registador de Saída): Armazena o resultado da operação executada pela ULA.
- memoria[2] -> X (Registador de Entrada X): Armazena o operando X em Hexadecimal.
- memoria[3] -> Y (Registador de Entrada Y): Armazena o operando Y em Hexadecimal.
- memoria[4 a 99] -> Área de Programa: Guarda os opcodes hexadecimais (.hex) vindos do PC.
- Mapeamento dos LEDs de Saída:
- Pino 13 -> Bit 3 (F3 - MSB)
- Pino 12 -> Bit 2 (F2)
- Pino 11 -> Bit 1 (F1)
- Pino 10 -> Bit 0 (F0 - LSB)

*/

// MOTIVO DE EXISTIR: Definição dos pinos digitais ligados aos LEDs da ULA conforme a Figura 1 do relatório.

const int PINO_F3 = 13; // Bit F3 (Mais Significativo / MSB)

const int PINO_F2 = 12; // Bit F2

const int PINO_F1 = 11; // Bit F1

const int PINO_F0 = 10; // Bit F0 (Menos Significativo / LSB)

// MOTIVO DE EXISTIR: Vetor que simula a memória RAM e os registradores unificados da arquitetura (100 posições).

String memoria[100];

// MOTIVO DE EXISTIR: Guarda o total de instruções efetivamente carregadas para controlar o ciclo e o DUMP.

int totalInstrucoes = 0;

// Protótipos das funções auxiliares

void carregarPrograma();

void executarPrograma();

int hexParaInt(char c);

int calcularULA(int A, int B, int S);

void atualizarLEDs(int valor);

void mostrarDumpMemoria(const char* mensagem);

void setup() {

// MOTIVO DE EXISTIR: Inicializa a comunicação serial a 9600 baud para receber as instruções geradas pelo software em C++.

```
Serial.begin(9600);
```

```
// MOTIVO DE EXISTIR: Configura os pinos 10 a 13 como saídas digitais para controlo dos LEDs.
```

```
pinMode(PINO_F3, OUTPUT);
```

```
pinMode(PINO_F2, OUTPUT);
```

```
pinMode(PINO_F1, OUTPUT);
```

```
pinMode(PINO_F0, OUTPUT);
```

```
// MOTIVO DE EXISTIR: Assegura que todos os LEDs iniciam desligados.
```

```
atualizarLEDs(0);
```

```
// MOTIVO DE EXISTIR: Limpa as 100 posições da memória atribuindo strings vazias.
```

```
for (int i = 0; i < 100; i++) {
```

```
  memoria[i] = "";
```

```
}
```

```
// MOTIVO DE EXISTIR: Inicialização dos registradores da máquina conforme exigido no enunciado.
```

```
memoria[0] = "4"; // PC inicia no índice 4 (início da área de programa)
```

```
memoria[1] = "0"; // Registrador W zerado
```

```
memoria[2] = "0"; // Registrador X zerado
```

```
memoria[3] = "0"; // Registrador Y zerado
```

```
Serial.println("");
```

```
Serial.println(" ULA DE 4 BITS - ARDUINO PRONTO PARA CARGA ");
```

```
Serial.println("");
```

```
Serial.println("Cole e envie o conteúdo do ficheiro testeula.hex:");
```

```
}
```

```
void loop() {
```

```
// MOTIVO DE EXISTIR: Aguarda a recepção do programa via Serial quando o vetor de memória ainda está vazio.
```

```
if (Serial.available() > 0 && totalInstrucoes == 0) {
```

```
  carregarPrograma();
```

```
if (totalInstrucoes > 0) {
```

```
  Serial.println("\n--- Carga do Vetor Concluída! ---");
```

```
  mostrarDumpMemoria("Carga do vetor:");
```

```
// MOTIVO DE EXISTIR: Cumpre a exigência de perguntar ao utilizador antes de iniciar a execução.
```

```
Serial.println("\nDeseja executar o programa? (s/n)");
```

```
while (!Serial.available()) {
```

```
  // Loop de espera bloqueante até o utilizador responder na Serial
```

```
}
```

```

char opcao = Serial.read();
if (opcao == 's' || opcao == 'S') {
Serial.println("\nIniciando execução instrução por instrução...\n");
executarPrograma();
} else {
Serial.println("\nExecução cancelada. Reinicie o Arduino para efetuar nova carga.");
}
}
}
}
}

```

```

/**

```

- MOTIVO DE EXISTIR: Lê a sequência enviada pela Serial (ex: "C6B A3E A34 D35") e armazena
- cada instrução de 3 caracteres na memória a partir da posição 4.

```

*/

```

```

void carregarPrograma() {
int indicePrograma = 4;
String bufferEntrada = Serial.readString();

```

```

// Normaliza quebras de linha e retornos de carro para espaços

```

```

bufferEntrada.replace("\r", " ");
bufferEntrada.replace("\n", " ");

```

```

int inicio = 0;
while (inicio < bufferEntrada.length()) {
while (inicio < bufferEntrada.length() && bufferEntrada.charAt(inicio) == ' ') {
inicio++;
}
if (inicio >= bufferEntrada.length()) break;

```

```

int fim = bufferEntrada.indexOf(' ', inicio);
if (fim == -1) fim = bufferEntrada.length();

```

```

String instrucao = bufferEntrada.substring(inicio, fim);
instrucao.trim();

```

```

// Processa instruções de 3 dígitos hexadecimais no formato [X][Y][S] gerados pelo seu código C++
if (instrucao.length() >= 3) {
instrucao = instrucao.substring(0, 3);
instrucao.toUpperCase();

```

```
memoria[indicePrograma] = instrucao;  
indicePrograma++;  
totalInstrucoes++;
```

```
if (indicePrograma >= 100) break; // Impede o extravasamento da memória de 100 posições  
}  
inicio = fim + 1;  
}  
}
```

```
/**
```

- MOTIVO DE EXISTIR: Realiza o ciclo de Busca, Decodificação e Execução.
- Utiliza o PC (posição 0) como ponteiro para ler a instrução atual e atualizar os registradores X, Y e W.

```
*/
```

```
void executarPrograma() {  
    int pc = memoria[0].toInt(); // Lê a posição atual a partir do registrador PC (memoria[0])
```

```
while (pc < (4 + totalInstrucoes)) {  
    String instrucao = memoria[pc];  
    if (instrucao.length() < 3) break;
```

```
// Decodifica os dígitos Hexadecimais da instrução [X][Y][S]  
char charX = instrucao.charAt(0);  
char charY = instrucao.charAt(1);  
char charS = instrucao.charAt(2);
```

```
int valX = hexParaInt(charX);  
int valY = hexParaInt(charY);  
int valS = hexParaInt(charS);
```

```
// MOTIVO DE EXISTIR: Atualiza os registradores X (memoria[2]) e Y (memoria[3]) na memória  
memoria[2] = String(charX);  
memoria[3] = String(charY);
```

```
// MOTIVO DE EXISTIR: Executa o cálculo lógico na ULA  
int resultadoW = calcularULA(valX, valY, valS);
```

```
// MOTIVO DE EXISTIR: Atualiza o resultado em W (memoria[1]) convertido para Hexadecimal  
memoria[1] = String(resultadoW, HEX);
```

```
memoria[1].toUpperCase();
```

```
// MOTIVO DE EXISTIR: Incrementa o PC para apontar para a próxima instrução e atualiza a memoria[0]  
pc++;  
memoria[0] = String(pc);
```

```
// MOTIVO DE EXISTIR: Envia o valor do registrador W para os 4 LEDs de saída  
atualizarLEDs(resultadoW);
```

```
// MOTIVO DE EXISTIR: Requisito do trabalho para exibir o estado completo da memória (DUMP) a cada  
ciclo  
mostrarDumpMemoria("Após a execução da instrução:");
```

```
// MOTIVO DE EXISTIR: Pausa obrigatoriamente 4 segundos entre cada instrução para acompanhamento  
visual nos LEDs  
delay(4000);  
}
```

```
Serial.println("\n=== Fim da Execução do Programa ===");  
}
```

```
/**
```

- MOTIVO DE EXISTIR: Converte caracteres em hexadecimal ('0'-'F') para os seus valores numéricos correspondentes (0-15).

```
*/  
int hexParaInt(char c) {  
    if (c >= '0' && c <= '9') return c - '0';  
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;  
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;  
    return 0;  
}
```

```
/**
```

- MOTIVO DE EXISTIR: Implementa a tabela de verdade de 16 instruções da ULA (Figura 2 do relatório).
- O valor de S (0 a 15) corresponde exatamente ao índice operacao[valor] - 1 calculado no seu C++.

```
*/  
int calcularULA(int A, int B, int S) {  
    int F = 0;
```

```

switch (S) {
case 0x0: F = (~A); break; // 0 -> nA (A')
case 0x1: F = ~(A | B); break; // 1 -> AoBn ((A + B)')
case 0x2: F = (~A) & B; break; // 2 -> nAeB (A' . B)
case 0x3: F = 0; break; // 3 -> zeroL (0)
case 0x4: F = ~(A & B); break; // 4 -> AeBn ((A . B)')
case 0x5: F = (~B); break; // 5 -> nB (B')
case 0x6: F = A ^ B; break; // 6 -> AxB (A' . B + A . B')
case 0x7: F = A & (~B); break; // 7 -> AenB (A . B')
case 0x8: F = (~A) | B; break; // 8 -> nAoB (A' + B)
case 0x9: F = ~(A ^ B); break; // 9 -> AxBn (A . B + A' . B')
case 0xA: F = B; break; // A -> copiaB (B)
case 0xB: F = A & B; break; // B -> AeB (A . B)
case 0xC: F = 0xF; break; // C -> umL (1 Lógico / 1111)
case 0xD: F = A | (~B); break; // D -> AonB (A + B')
case 0xE: F = A | B; break; // E -> AoB (A + B)
case 0xF: F = A; break; // F -> copiaA (A)
}

```

```

// MOTIVO DE EXISTIR: O operador de máscara '& 0x0F' restringe o resultado obrigatoriamente a 4 bits.
return F & 0x0F;
}

```

/**

- MOTIVO DE EXISTIR: Ativa ou desativa os pinos digitais 10 a 13 segundo os bits correspondentes de W.

*/

```

void atualizarLEDs(int valor) {
digitalWrite(PINO_F3, (valor & 0x08) ? HIGH : LOW); // Bit 3 -> Pino 13
digitalWrite(PINO_F2, (valor & 0x04) ? HIGH : LOW); // Bit 2 -> Pino 12
digitalWrite(PINO_F1, (valor & 0x02) ? HIGH : LOW); // Bit 1 -> Pino 11
digitalWrite(PINO_F0, (valor & 0x01) ? HIGH : LOW); // Bit 0 -> Pino 10
}

```

/**

- MOTIVO DE EXISTIR: Exibe a memória formatada exatamente conforme o exemplo do relatório:
- Formatado como: ->| PC | W | X | Y | INST1 | INST2 | ...

/

```

void mostrarDumpMemoria(const char mensagem) {

```

```
Serial.println(mensagem);  
Serial.print(" ->| ");
```

```
// Exibe o estado atual dos registradores principais nas posições 0, 1, 2 e 3
```

```
Serial.print(memoria[0]); Serial.print(" | ");  
Serial.print(memoria[1]); Serial.print(" | ");  
Serial.print(memoria[2]); Serial.print(" | ");  
Serial.print(memoria[3]); Serial.print(" | ");
```

```
// Exibe apenas as posições do vetor onde existem instruções ativas
```

```
for (int i = 4; i < (4 + totalInstrucoes); i++) {  
  Serial.print(memoria[i]);  
  Serial.print(" | ");  
}  
Serial.println();  
}
```